

LiteLLM Integration Guide: Cost & Usage Tracking, Both Directions (2026 Edition)

TL;DR

- **LiteLLM (MIT-licensed, ~v1.88.x on PyPI as of June 2026) is the right choice for both consuming and providing metered LLM access in a self-hosted Apache-2.0-preferring stack like mindX** — deploy the open-source Proxy/Gateway container backed by your existing PostgreSQL 16 + Redis. Spend tracking, virtual keys, budgets, rate limits, Prometheus metrics, customer/end-user billing endpoints, and the Admin UI are all in the free MIT build; only SSO/SAML beyond 5 users, audit logs, JWT/SCIM auth, and certain guardrail callbacks require LiteLLM Enterprise.
- **For CONSUMING (internal cost attribution):** map each BDI agent / cognitive subsystem / persona to a `(team_id)` + virtual key + request `(tags)`, propagate `(user)`/`(end_user)` for per-agent spend, and stream cost events from a custom `CustomLogger.async_log_success_event` callback (reading `(kwargs["response_cost"])`) into NATS JetStream for your Dash/Textual dashboards.
- **For PROVIDING (metered marketplace access):** issue per-customer virtual keys with `(max_budget)`/`(budget_duration)` and TPM/RPM limits, track per-customer spend via `(/customer)` endpoints, and gate x402 (Algorand/`(x402-avm)`) micropayments using either a `(custom_auth)` hook that verifies a payment receipt before the request proceeds, or a prepaid-credit model where on-chain settlement tops up LiteLLM budgets via `(/customer/update)` or `(/key/update)`.

Key Findings

Versioning, health, and the March 2026 supply-chain incident

LiteLLM is actively maintained by BerriAI with weekly stable releases. As of June 2026 the PyPI package is in the 1.88.x line (1.88.1 released Jun 9, 2026), with GitHub stable tags around v1.84–v1.85. The project moved to clean SemVer-style versioning starting 1.84.0: MINOR bumps weekly, PATCH reserved for hotfixes, and `(:latest)` is the canonical rolling Docker tag (the legacy `(main-stable)` tag is planned to stop being published June 30, 2026). Adoption is very high — pypistats.org reports **96,830,964 downloads in the last month**, and the Cloud Security Alliance pegs it at "approximately 95 million monthly PyPI downloads as of March 2026." `(LiteLLM)`

Critical security note: On **March 24, 2026**, per the Cloud Security Alliance, "threat actor TeamPCP published backdoored versions of LiteLLM (v1.82.7 and v1.82.8) to PyPI" — the fourth wave of a TeamPCP campaign that earlier compromised Aqua Security's Trivy scanner, whose stolen credentials were then used against LiteLLM's CI/CD publishing pipeline. Endor Labs (via CyberInsider) confirmed **version 1.82.6 was the last clean pre-incident release**; only ~12 lines of

obfuscated credential-stealer code were injected into `litellm/proxy/proxy_server.py` during the wheel build, with roughly a 5-hour exposure window before PyPI quarantine. v1.83.0 was the first clean release published afterward (via a hardened CI/CD v2 pipeline). **Always pin to a known-good content tag and verify the cosign signature** — never use unpinned `pip install litellm` or rolling tags in production.

SDK vs Proxy — pick Proxy for tracking/metering

- **Python SDK** (`litellm.completion` / `acompletion`): direct library integration, Router with retry/fallback/load-balancing, application-level cost tracking via `completion_cost()` and callbacks. Best for embedded use where you control the calling code.
- **Proxy Server / LLM Gateway** (`litellm[proxy]`): centralized OpenAI-compatible gateway with authentication, virtual keys, multi-tenant spend management, budgets, guardrails, caching, and an Admin UI. **For both cost attribution across many agents AND metering external customers, the Proxy is the correct deployment mode** — it persists spend to PostgreSQL and centralizes key/budget logic so application code stays unchanged.

Both are MIT and async-first. Python requirement is `>=3.10, <3.14`, so mindX's Python 3.12 is fully supported. `PyPI`

Async, router, caching

The SDK exposes `acompletion`, async streaming, and async embeddings. The Router supports `simple-shuffle` (recommended default), `least-busy`, `usage-based-routing-v2`, and `latency-based-routing`, plus fallbacks, content-policy fallbacks, and per-exception retry/allowed-fail policies. Caching supports in-memory, Redis, Redis Cluster/Sentinel, S3, GCS, disk, and **semantic caching** (Redis-semantic via RediSearch, or Qdrant). Note: Redis **semantic** caching requires Redis Stack with the RediSearch module loaded — a common deployment gotcha, since many managed Redis offerings don't ship it. mindX already runs Qdrant, so `qdrant_semantic_cache` is a natural fit. A `DualCache` (L1 in-memory + L2 Redis) with a `RedisCircuitBreaker` is used internally for distributed state.

Cost-tracking mechanics

- `completion_cost(response)` returns total USD cost for a call; `cost_per_token(model, prompt_tokens, completion_tokens)` returns input/output cost separately; `token_counter()` counts tokens (tiktoken for OpenAI, bundled tokenizers for others, plus custom tokenizers).
- Pricing comes from `model_prices_and_context_window.json` (auto-updated via GitHub Actions; a backup copy ships in the package). Override per-model in proxy config via `model_info` keys (`input_cost_per_token`, `output_cost_per_token`, `cache_creation_input_token_cost`, `cache_read_input_token_cost`),

- `input_cost_per_second` for SageMaker, `input_cost_per_image`, `output_cost_per_reasoning_token`, `input_cost_per_audio_token`, and the `*_above_200k_tokens` tiered variants). (LiteLLM)
- **Self-hosted/custom models:** register pricing with `litellm.register_model({...})` in the SDK, or set `model_info` costs in proxy config. To make a free/on-prem model bypass ALL budget checks, set **both** `input_cost_per_token: 0` and `output_cost_per_token: 0` explicitly. To avoid network calls to the hosted cost map behind a firewall, point `LITELLM_MODEL_COST_MAP_URL` at a local copy.
- Streaming cost is tracked (the success callback receives `response_cost` once the stream completes). Embedding, image, and audio costs are tracked using the corresponding cost-map fields and per-token-detail breakouts.

Proxy spend tracking, hierarchy, budgets

- Spend is written to the `LiteLLM_SpendLogs` table; per-key running spend is tracked in `LiteLLM_VerificationToken` (verification token table). The spend log row includes `api_key` (hashed), `user`, `team_id`, `request_tags`, `end_user`, `model_group`, `api_base`, `spend`, and token counts. (LiteLLM) (LiteLLM)
- Hierarchy: **Organization** → **Team** → **User** → **Virtual Key**, plus **End-User/Customer** (the `user` field) and **Budget** objects. Constraints at higher levels are global ceilings; lower levels give granular control. Budgets support `max_budget` (hard limit, rejects requests), `soft_budget` (alert-only), `budget_duration` (e.g. `24h`, `30d`, `30s`), multi-window budgets (`budget_limits` array), and model-specific budgets (`model_max_budget`). (LiteLLM)
- Endpoints: `/key/generate`, `/key/update` (incl. `temp_budget_increase`), `/user/new`, `/team/new`, `/customer/new`, `/budget/new`, `/spend/logs`, `/global/spend/report` (supports `group_by=team`), `/global/spend/reset`, `/customer/info?end_user_id=`, `/user/daily/activity`. Spend updates are buffered (`proxy_batch_write_at`, default 60s) and use a Redis buffer (`use_redis_transaction_buffer`) for atomic cross-instance enforcement. (LiteLLM) (LiteLLM)
- Every response carries the `x-litellm-response-cost` header with the calculated cost. (LiteLLM)

Database

The Proxy requires PostgreSQL for virtual keys, spend logs, teams/users, and the UI (`database_url` in `general_settings`, or `DATABASE_URL`). It uses Prisma; schema migrations are shipped via the `litellm-proxy-extras` package and applied on upgrade. mindX's existing PostgreSQL 16 works directly — use the `litellm-database` Docker image variant (ships pre-generated Prisma binaries). Connection pooling is set via `database_connection_pool_limit` (default 10 per worker); total connections = limit × workers × instances, so size carefully (a

PgBouncer sidecar in transaction-pooling mode is a known pattern for high concurrency).

LiteLLM

Callbacks & observability (Prometheus is OSS again as of 2026)

- **Custom callbacks:** subclass `litellm.integrations.custom_logger.CustomLogger` and implement `async_log_success_event(self, kwargs, response_obj, start_time, end_time)`; read cost via `kwargs.get("response_cost")`. Register via `litellm.callbacks = [handler]` (SDK) or `litellm_settings: callbacks: custom_callbacks.proxy_handler_instance` (proxy). Proxy-only hooks include `async_pre_call_hook` (modify/reject requests), `async_moderation_hook`, `async_post_call_failure_hook`, and `async_post_call_response_headers_hook`. Callbacks can be loaded from S3/GCS URLs.
- **Prometheus:** enable with `litellm_settings: callbacks: ["prometheus"]`, scrape `/metrics`. **As of LiteLLM v1.80.0 (late 2025), Prometheus metrics are available in the free open-source build and no longer require an Enterprise license** — the v1.80.0 release note states they are "now available in the open-source version of LiteLLM ... without requiring an enterprise license." This reverses a Sept 15, 2024–2025 period when Prometheus metrics were Enterprise-gated (announced in GitHub Discussion #5163 at "\$250/mo"), which is why Datadog's integration docs and several blogs still say "enterprise only" — those are **stale**. Historically a `LITELLM_LICENSE` check ran at proxy startup when the prometheus callback was set (see Issue #7817 / PR #8492); that gating was removed in v1.80.0+. Key metrics: `litellm_spend_metric`, `litellm_total_tokens_metric`, `litellm_input_tokens_metric`, `litellm_output_tokens_metric`, `litellm_remaining_team_budget_metric`, `litellm_remaining_api_key_budget_metric`, `litellm_api_key_max_budget_metric`, `litellm_remaining_api_key_requests_for_model`, plus token-detail counters (`litellm_input_cached_tokens_metric`, `litellm_output_reasoning_tokens_metric`, etc.) and `litellm_in_flight_requests`. The `/metrics` endpoint is **unauthenticated by default** — set `require_auth_for_metrics_endpoint: true` (free OSS setting) or restrict at the network layer (a known issue otherwise lets any valid key scrape all metrics). For multiple workers set `PROMETHEUS_MULTIPROC_DIR`. End-user labels are off by default to limit cardinality. `LiteLLM + 6`
- **OpenTelemetry:** `callbacks: ["otel"]` with standard `OTEL_EXPORTER_OTLP_*` env vars; emits one trace per request (HTTP → auth → guardrails → LLM call → DB writes) following GenAI semantic conventions, with INTERNAL child spans for router/auth/redis/postgres timing. Works with any OTLP backend. Pass a `traceparent` header to nest LiteLLM spans under a parent agent trace (ideal for mindX's structlog + OpenTelemetry stack). Note: the official `litellm-database` image historically shipped without `opentelemetry-`

`instrumentation`), so OTEL metrics/logs export (vs traces) may need an extra dependency.

`(LiteLLM + 3)`

- **Billing/observability integrations:** Langfuse (incl. `(langfuse_otel)`), Helicone, Lunary, MLflow, Datadog, **OpenMeter** (`(callbacks: ["openmeter"])`), and **Lago** (`(callbacks: ["lago"])`), with `(LAGO_API_CHARGE_BY) = (end_user_id)/(user_id)/(team_id)` — both OpenMeter and Lago are open-source usage-based billing engines that integrate with Stripe. `(LiteLLM)` `(LiteLLM)`

Tagging & attribution

Requests can carry `(metadata.tags)` (or `(x-litellm-tags)` header, or config-based tags) that land in `(request_tags)`. The `(user)` field (or `(x-litellm-customer-id)/(x-litellm-end-user-id)` headers) attributes spend to an end-user/customer. Team/user IDs propagate from the virtual key. **Security caveat:** if you give end-users keys, you must set `(user_id)` when creating their keys — otherwise a client passing its own `(user)` value can shift spend off its own key. User-Agent is tracked as a tag by default (useful for distinguishing Claude Code, Gemini CLI, etc.). `(LiteLLM + 2)`

Alerting

`(general_settings: alerting: ["slack"])` with `(SLACK_WEBHOOK_URL)`; alert types include `(budget_alerts)`, `(spend_reports)`, `(llm_exceptions)`, `(llm_too_slow)`, `(llm_requests_hanging)`, `(db_exceptions)`, `(daily_reports)`, `(cooldown_deployment)`, `(outage)` alerts. Per-type Slack channels via `(alert_to_webhook_url)`. Multi-threshold budget alerts via `(alerting_thresholds: [0.5, 0.75, 0.9, 1.0])`. Generic webhook alerts emit `(spend_tracked)`, `(budget_crossed)`, `(threshold_crossed)` (85%/95%), and `(projected_limit_exceeded)` events with an `(event_group)` of `(customer)/(internal_user)/(key)/(team)/(proxy)`. Email (SendGrid/Resend/SMTP) and PagerDuty are also supported. **Known issue:** with multiple replicas, alerts/spend reports are emitted per-pod (duplicates) unless deduplicated. `(LiteLLM)`

Providing: customers, multi-tenancy, billing

- `(/customer/new)` + `(/budget/new)` create pricing tiers (e.g. a `(free-tier)` budget with `(max_budget)`, `(tpm_limit)`, `(rpm_limit)`) assignable by `(budget_id)`. `(litellm.max_end_user_budget_id)` can point all implicit end-users to a default budget (with a known bug where implicitly-created end-users may not persist the default `budget_id` — set it explicitly via `(/customer/new)`). `(LiteLLM)`
- Per-key model access control (`(models: [...])`), model groups, team-based isolation, and key/customer rate limits (TPM/RPM, including input/output/total TPM type) are all OSS.
- Usage-based billing: export spend via `(/spend/logs)` / `(/global/spend/report)`, or push real-time events to OpenMeter/Lago (→ Stripe). For crypto, drive billing from the success-callback `(response_cost)`.

Custom auth & x402/crypto micropayments

- `(general_settings: custom_auth: custom_auth.user_api_key_auth)` lets you supply an `async (user_api_key_auth(request, api_key) -> UserAPIKeyAuth)` function. By default LiteLLM does NOT run its standard budget/model checks after custom auth returns; set `(custom_auth_run_common_checks: true)` to enforce them. JWT/OIDC auth `(enable_jwt_auth)` maps token claims to user/team/org/end-user fields and is partly Enterprise.
- **x402 on Algorand via `(x402-avm)` (GoPlausible, Algorand Foundation):** the Python package `((pip install x402-avm), imports (from x402...))` provides an `httpx (AsyncBaseTransport) ((x402AsyncTransport), async via (x402Client))` and a `requests (HTTPAdapter) ((x402ClientSync))` that intercept HTTP 402, sign an Algorand ASA/USDC transfer (atomic group, optional fee abstraction where the facilitator pays gas), and retry with a `(PAYMENT-SIGNATURE)` header. The protocol joined the Linux Foundation in April 2026; V2 (Dec 2025) added multi-chain support and **wallet sessions** (authenticate once, settle accumulated usage periodically) which make per-request LLM micropayments practical.
- **Two integration patterns** (no off-the-shelf x402+LiteLLM bridge exists yet — these are reference architectures):
 1. **Gate-then-forward (custom auth / sidecar):** Put an x402-aware FastAPI middleware or sidecar in front of the LiteLLM Proxy. It returns 402 with payment terms, verifies the on-chain payment via a facilitator, then forwards the request to the Proxy with an internal virtual key. Or implement the check inside a LiteLLM `(custom_auth)` hook / `(async_pre_call_hook)` that validates a payment receipt header before the call proceeds.
 2. **Prepaid credit / top-up (recommended for high frequency):** Customer pays on-chain (Algorand x402); a settlement listener converts the USDC payment into a LiteLLM budget top-up by calling `(/customer/update)` (or `(/key/update)` with `(temp_budget_increase)`). LiteLLM then enforces spend natively and you get per-customer accounting for free. This aligns with x402 V2 wallet sessions and avoids an on-chain round-trip per request. Public reference implementations of the gate-then-forward pattern exist on other chains (e.g. a Solana `(x402-ai)` multi-LLM gateway with on-chain receipts and automatic refunds; Bankr's x402 Cloud LLM gateway on Base) — the same architecture maps onto Algorand via `(x402-avm)`.

Admin UI & Enterprise boundary (precise, 2026)

Free OSS (MIT): OpenAI-compatible gateway; virtual keys; spend tracking (request/tag/key/user/team/org/end-user); budgets & rate limits at every level; request/response logs; fallbacks & load balancing; **Prometheus metrics** (as of v1.80.0+); custom callbacks; OTEL; the Admin UI for key/team/budget/model management; key rotation config; caching; **SSO free for**

up to 5 users. Requires LiteLLM Enterprise (license key): SSO/SAML beyond 5 users (Okta/Azure AD/Google/OIDC/SAML); SCIM; JWT auth; audit logs; fine-grained/group-based access delegation; custom Swagger/email/UI branding; max request/response size limits; blocked-user lists; and several built-in guardrail callbacks (`llmguard_moderations`), `llamaguard_moderations`, `hide_secrets`, `openai_moderations`, `google_text_moderation`, `lakera_prompt_injection`, `aporia_prompt_injection`). The OSS guardrail framework itself (custom guardrails + Presidio PII masking) is free. Professional support/SLA is Enterprise-only.

Pricing: TrueFoundry cites "Enterprise Basic: \$250/month ... Enterprise Premium: \$30,000/year," but LiteLLM's own enterprise page states pricing "is based on usage. Contact us for a quote" and that "SSO is free for up to 5 users" — the dollar figures are unpublished/negotiated (the AWS Marketplace listing requires a Private Offer via sales@berri.ai), so treat them as ballpark and confirm with the vendor.

Security, scaling, performance

- **Keys:** `LITELLM_MASTER_KEY` (admin root, must start with `sk-`) and `LITELLM_SALT_KEY` (encrypts stored provider keys — **never rotate it** or you lose access to stored credentials). Automatic virtual-key rotation via `LITELLM_KEY_ROTATION_ENABLED=true` + `auto_rotate`/`rotation_interval` on keys (DB required). Store secrets in env/`.env` (mount read-only), never in the image; integrates with Vault/cloud secret managers. `LiteLLM`
- **Scaling:** stateless proxy scales horizontally; run **one Uvicorn worker per pod/container and add more pods** rather than more workers per pod (predictable latency, hitless rolling restarts). Use Gunicorn (`--run_gunicorn`) only when packing multiple workers per container, with `--max_requests_before_restart` to recycle workers (counters memory growth). Redis is **required** for distributed rate-limit/budget state and shared auth cache (`enable_redis_auth_cache: true`) across replicas; 1000+ RPS deployments need Redis to avoid PostgreSQL connection exhaustion. Shared health-check state (`use_shared_health_check: true`) avoids duplicate provider health calls across pods.
- **Performance:** With `USE_AIOHTTP_TRANSPORT=True` (now the recommended/default-track transport), the v1.71.1-stable release notes state "LiteLLM can now scale to 200 RPS per instance with a 40ms median latency overhead ... This change doubles the RPS." LiteLLM's "Achieving Sub-Millisecond Proxy Overhead" blog reports the proxy "can be stress-tested at 1,000 QPS with no failures and can scale up to 5,000 QPS without failures on a 4-CPU, 8-GB RAM single instance setup" (responding to an earlier TensorZero benchmark where LiteLLM failed at ~1,000 QPS), alongside a ~30% overhead reduction from middleware optimizations. Every response includes `x-litellm-overhead-duration-ms`. Don't run DEBUG logging in production with large payloads (synchronous `json.dumps(indent=4)` can add seconds). Production baseline: ≥4 CPU cores, ≥8 GB RAM.

Alternatives — when LiteLLM Proxy is/isn't right

- **OpenRouter:** hosted aggregator; great for zero-ops multi-provider access but it's a third party in the request path and not self-hostable — wrong for mindX's self-hosted, data-control requirements.
- **Portkey:** managed SaaS (self-host option exists) with built-in semantic caching, observability dashboard, guardrails, SOC2/HIPAA pre-certified — faster time-to-production for mid-size teams without gateway DevOps, but SaaS-first.
- **Kong AI Gateway:** strong enterprise API-gateway lineage; heavier and more infra-centric; reasonable if you already run Kong.
- **Helicone:** observability-first gateway/proxy; excellent logging/analytics, lighter on multi-tenant budgeting/virtual-key governance than LiteLLM.
- **Bifrost (Maxim AI, Apache 2.0, Go):** positions as a high-performance LiteLLM alternative — Maxim AI's GitHub states "in sustained 5,000 RPS benchmarks, the gateway added only 11 μ s of overhead per request" (11 μ s on t3.xlarge, 59 μ s on t3.medium), and claims "~9.5x faster, ~54x lower P99 latency, and uses 68% less memory than LiteLLM," with a native MCP gateway, hierarchical governance, and a drop-in LiteLLM-compatible API. Worth evaluating if Python-proxy throughput becomes a ceiling and you want Apache-2.0 licensing (vendor-published benchmarks; validate independently).
- **Verdict for mindX:** LiteLLM Proxy (OSS) is the best fit — MIT license, self-hosted, Podman-friendly, OpenAI-compatible, with the deepest native cost-attribution + customer-metering feature set among open-source options. The main watch-items are Python-proxy throughput at extreme scale and the operational burden of running PostgreSQL+Redis (which mindX already operates).

Details

Reference `config.yaml` (consume + provide)

```
yaml
```



```

model_list:
  - model_name: gpt-5
    litellm_params:
      model: openai/gpt-5
      api_key: os.environ/OPENAI_API_KEY
  - model_name: claude-sonnet
    litellm_params:
      model: anthropic/claude-sonnet-4-5
      api_key: os.environ/ANTHROPIC_API_KEY
      rpm: 4
      tpm: 25000
  - model_name: mindx-local # self-hosted model, free → bypass budgets
    litellm_params:
      model: openai/mindx-local
      api_base: http://vllm:8000/v1
      api_key: none
    model_info:
      input_cost_per_token: 0 # both explicitly 0 to skip budget checks
      output_cost_per_token: 0

litellm_settings:
  callbacks: ["prometheus", "otel", "custom_callbacks.nats_cost_handler"]
  cache: true
  cache_params:
    type: qdrant-semantic # mindX already runs Qdrant
    qdrant_semantic_cache_embedding_model: text-embedding-ada-002
  require_auth_for_metrics_endpoint: true
  redact_user_api_key_info: true

general_settings:
  master_key: os.environ/LITELLM_MASTER_KEY
  database_url: os.environ/DATABASE_URL # mindX PostgreSQL 16
  alerting: ["slack"]
  alerting_thresholds: [0.5, 0.75, 0.9, 1.0]
  proxy_batch_write_at: 60
  database_connection_pool_limit: 10
  custom_auth: custom_auth.x402_user_api_key_auth # x402 gate (PROVIDE mode)
  custom_auth_run_common_checks: true

```

NATS cost-event callback (CONSUME → Dash/Textual)

```
python
```

```

# custom_callbacks.py
from litellm.integrations.custom_logger import CustomLogger
import nats, json, structlog

log = structlog.get_logger()

class NATSCostHandler(CustomLogger):
    def __init__(self):
        self._nc = None

    async def _ensure(self):
        if self._nc is None:
            self._nc = await nats.connect("nats://nats:4222")
        return self._nc

    async def async_log_success_event(self, kwargs, response_obj, start_time, end_time):
        try:
            md = kwargs.get("litellm_params", {}).get("metadata", {}) or {}
            event = {
                "cost_usd": kwargs.get("response_cost", 0.0),
                "model": kwargs.get("model"),
                "team_id": md.get("user_api_key_team_id"),
                "agent": md.get("tags", []),          # e.g. ["agent:bdi-planner"]
                "end_user": kwargs.get("user"),
                "total_tokens": getattr(getattr(response_obj, "usage", None), "total_tokens", 0),
                "latency_s": (end_time - start_time).total_seconds(),
            }
            nc = await self._ensure()
            # Dash diagnostics + Textual TUI subscribe to these subjects
            await nc.publish("mindx.cost.events", json.dumps(event).encode())
        except Exception as e:
            # never break the request path
            log.warning("nats_cost_publish_failed", error=str(e))

nats_cost_handler = NATSCostHandler()

```

Calling the proxy from async FastAPI (openai SDK)

python

```
from openai import AsyncOpenAI
client = AsyncOpenAI(api_key="sk-agent-bdi-planner-key", base_url="http://litellm:4000")
resp = await client.chat.completions.create(
    model="claude-sonnet",
    messages=[{"role": "user", "content": "..."}],
    user="customer-123", # end-user attribution
    extra_body={"metadata": {"tags": ["agent:bdi-planner", "subsystem:mind"]}},
)
```

Issuing a metered customer key (PROVIDE)

```
bash

# 1. create a pricing-tier budget
curl -X POST 'http://litellm:4000/budget/new' -H 'Authorization: Bearer $MASTER' \
-H 'Content-Type: application/json' \
-d '{"budget_id":"marketplace-prepaid","max_budget":10,"tpm_limit":25000,"rpm_limit":1000000}'

# 2. create the customer on that tier
curl -X POST 'http://litellm:4000/customer/new' -H 'Authorization: Bearer $MASTER' \
-H 'Content-Type: application/json' \
-d '{"user_id":"algo-wallet-XYZ","budget_id":"marketplace-prepaid"}'

# 3. top up after an on-chain x402 settlement
curl -X POST 'http://litellm:4000/customer/update' -H 'Authorization: Bearer $MASTER' \
-H 'Content-Type: application/json' \
-d '{"user_id":"algo-wallet-XYZ","max_budget":25}'
```

Podman / Quadlet deployment

Use the `litellm-database` image (pre-generated Prisma binaries). A `.container` Quadlet unit with a healthcheck on `/health/readiness`, depending on Postgres and Redis units:

```
ini
```

```
# litellm.container
[Container]
ContainerName=litellm
Image=ghcr.io/berriai/litellm-database:v1.85.0 # PIN a known-good tag (post-incident)
PublishPort=4000:4000
Volume=/etc/mindx/litellm/config.yaml:/app/config.yaml:ro
EnvironmentFile=/etc/mindx/litellm/.env # MASTER/SALT/DATABASE_URL/provider
Exec=--config /app/config.yaml --port 4000 --num_workers 1
HealthCmd=python3 -c "import urllib.request,sys,json; sys.exit(0 if json.load(urllib.request.urlopen('http://localhost:4000/health/readiness')).get('status','error')=='ok' else 1)"
HealthInterval=30s
[Unit]
Requires=postgres.service redis.service
After=postgres.service redis.service
[Install]
WantedBy=default.target
```

Health endpoints: `/health/readiness` (DB/cache status, unauthenticated), `/health/liveliness`, `/health` (live provider checks, auth required), `/health/backlog` (in-flight count). Run one worker per container and scale by adding replicas; point all replicas at shared Postgres + Redis.

Attribution model for the BDI platform

Recommended mapping for granular cost accounting:

- **Organization** = PYTHAI tenant (e.g. agenticplace marketplace vs internal mindX).
- **Team** = cognitive subsystem or agent class (Soul / Mind / Hands; or per-persona). Team budgets cap each subsystem.
- **Virtual Key** = per running agent instance (set `user_id` on creation so spend can't be spoofed). Enables per-agent budgets and rate limits.
- **Tags** = fine-grained dimensions (`agent:<id>`, `subsystem:mind`, `task:<type>`, `persona:<name>`) for roll-ups in `/global/spend/report` and Prometheus `custom_prometheus_metadata_labels`.
- **End-user** (`user`) = external marketplace consumer for the PROVIDE direction.

Decision table — which tracking/metering approach for which scenario

Scenario	Approach	Key mechanism
Embedded single-service cost tracking, no gateway	SDK + <code>completion_cost()</code> / success callback	<code>litellm.callbacks</code> , <code>kwargs["response_cost"]</code>

Scenario	Approach	Key mechanism
Per-agent / per-subsystem internal attribution	Proxy: Team per subsystem, key per agent, tags	<code>team_id</code> + <code>user_id</code> on key + <code>metadata.tags</code>
Real-time spend dashboards (Dash/Textual)	Custom <code>CustomLogger</code> → NATS JetStream	<code>async_log_success_event</code> publishes events
Ops/SRE metrics + Grafana	Prometheus callback (<code>/metrics</code>)	<code>litellm_spend_metric</code> , budget metrics (OSS)
Distributed tracing into mindX OTEL	<code>otel</code> callback + <code>traceparent</code> propagation	one trace/request, GenAI semconv
External customer metering (hard limits)	Per-customer virtual key + budget tier	<code>/key/generate</code> + <code>max_budget</code> + TPM/RPM
External metering (shared key, soft attribution)	<code>end_user</code> / <code>user</code> param + customer budgets	<code>/customer/new</code> + <code>x-litellm-customer-id</code>
Invoice generation / SaaS billing	OpenMeter or Lago callback → Stripe	<code>`callbacks: ["openmeter"]</code>
Pay-per-call crypto, no prepayment	x402 gate-then-forward via <code>custom_auth</code> /sidecar	verify 402 receipt before forwarding
High-frequency crypto access	x402 prepaid credit → budget top-up	settlement listener → <code>/customer/update</code>

Recommendations

Stage 1 — Stand up the gateway (CONSUME). Deploy the `litellm-database` Proxy container via Quadlet against mindX's PostgreSQL 16 + a Redis unit, pinned to a post-incident tag ($\geq$ v1.83.0; v1.82.6 was the last clean pre-incident release) with cosign verification. Configure provider keys via `EnvironmentFile`, set `master_key`/`salt_key`, enable `prometheus` + `otel` callbacks, and wire the NATS cost callback. Create one Team per cognitive subsystem and one virtual key per agent with `user_id` set. **Benchmark to advance:** stable spend rows in `LiteLLM_SpendLogs`, cost events flowing to NATS, and `x-litellm-overhead-duration-ms` < ~50ms p50.

Stage 2 — Dashboards & guardrails. Scrape `/metrics` (with `require_auth_for_metrics_endpoint: true`) into Prometheus/Grafana; render real-time spend in Dash and budget status in the Textual TUI from the NATS stream. Set soft budgets (alert) + hard `max_budget` per team and Slack/webhook alerts at 50/75/90/100%. **Threshold to revisit:** if

proxy overhead p95 climbs or you approach ~200 RPS/instance, ensure `USE_AIOHTTP_TRANSPORT` is on, add replicas (one worker each), and add a PgBouncer sidecar.

Stage 3 — Provide metered access (marketplace). Introduce `/customer` + `/budget` pricing tiers, per-customer keys with TPM/RPM and model allowlists, and push usage to OpenMeter or Lago for invoicing. **Decision point:** choose per-customer **virtual keys** (simplest, native hard budgets) over shared-key + `end_user` if you need hard per-customer enforcement.

Stage 4 — x402 micropayments. Start with the **prepaid-credit** pattern: an Algorand settlement listener (using `x402-avm`) tops up LiteLLM customer budgets via `/customer/update`. Only build the per-request gate-then-forward `custom_auth` path if you need true pay-per-call with no prepayment. Use x402 V2 wallet sessions to avoid an on-chain round-trip per request. **Threshold to change approach:** if on-chain latency/fees per request hurt UX, stay with prepaid top-ups.

Stage 5 — Decide on Enterprise. Stay on OSS unless/until you need SSO beyond 5 admins, audit logs for compliance, SCIM, or vendor SLA — then scope LiteLLM Enterprise (confirm pricing with vendor; figures are negotiated). If Python-proxy throughput becomes the bottleneck at marketplace scale, benchmark Bifrost (Apache 2.0, Go) as a drop-in.

Caveats

- **Supply-chain risk is real:** pin Docker/PyPI versions, verify cosign signatures, and treat gateway patching as recurring ops. Versions 1.82.7/1.82.8 were backdoored by TeamPCP on March 24, 2026 (1.82.6 was the last clean prior release).
- **Version-dependent feature boundaries:** the OSS-vs-Enterprise line moves between releases. Prometheus metrics are OSS as of v1.80.0+ (were Enterprise-gated Sept 2024–2025); third-party docs (Datadog, TrueFoundry) still say "enterprise only" and are stale. Re-verify at `docs.litellm.ai/docs/enterprise` before relying on any boundary. `LiteLLM`
- **Enterprise pricing (~\$250/mo Basic, ~\$30k/yr Premium) is not officially published** — LiteLLM's own page says "Contact us for a quote"; the dollar figures are third-party estimates, negotiated with the vendor.
- **End-user budget edge cases:** documented field name is `end_user_max_budget` (not `max_end_user_budget`), and implicitly-created end-users may not persist a default `budget_id` (open bug) — create customers explicitly via `/customer/new`. `GitHub`
- **No public, production x402+LiteLLM reference exists yet on Algorand specifically** — the patterns above are architecture recommendations synthesized from `x402-avm` capabilities and analogous Solana (`x402-ai`) / Base (Bankr x402 Cloud) LLM-gateway implementations; validate the facilitator/settlement flow on Algorand testnet first.
- **Semantic caching** requires Redis Stack + RediSearch (or Qdrant); plain managed Redis won't work for `redis-semantic`.
- **Multi-replica alerting** can duplicate Slack/spend-report messages; dedupe at the receiver or limit to one reporting pod.

- **Router async callback gotcha:** some historical versions had `Router.acompletion` not firing `CustomLogger` hooks reliably (Issue #8842) — test your cost callback end-to-end on the pinned version.
- The mindX/PYTHAI specifics (Parsec branding, ERC-8004 identity, exact chain mapping at agenticplace.pythai.net/allchain.html) were not independently verifiable from public LiteLLM sources; integrate per your internal architecture docs.